Forge

An Autonomous Multi-Agent Open Source Engineer

FastAPI · Python 3.12 | Next.js 15 · TypeScript | LangGraph · LangChain | PostgreSQL · Qdrant | tree-sitter · GitPython

Project Status Report

Generated 13 June 2026

Branch: **feature/live-agent-outputs**

Author: Prahar

1. Executive Summary

Forge is an autonomous software-engineering platform that behaves like a small engineering team. It connects to a GitHub repository, builds a semantic understanding of the codebase, analyzes an issue, plans an implementation, writes the code, validates and reviews its own work, and prepares a pull request — with minimal human intervention.

The guiding principle is **execution over conversation**: every feature must help the system **Think**, **Decide**, or **Act**. The product is built as a Turborepo monorepo combining a FastAPI backend, a Next.js 15 dashboard, and a LangGraph multi-agent workflow engine shared across reusable Python packages.

6	11	7 / 7	6	20+
CORE AGENTS	WORKFLOW NODES	CRITICAL PHASES DONE	PYTHON PACKAGES	REACT COMPONENTS

2. System Architecture

Forge is a pnpm + uv monorepo with two deployable apps and six shared packages. The LangGraph workflow is **application-agnostic**: graph nodes are pure functions that never touch the database or construct LLM clients — all dependencies are injected, and all repository data is pre-loaded into typed state by the backend's `BrainService`.

Layer	Component	Responsibility
apps/	`web` (Next.js 15)	Dashboard, repositories, agent center, live run detail, settings
	`api` (FastAPI)	Routes, services, repositories (DB layer), schemas, models, Alembic migrations
packages/	`agents`	11 LangGraph agent definitions (analyzer → PR generator)
	`workflows`	The compiled `StateGraph` and typed `SentinelState`
	`repository-analysis`	tree-sitter analyzer, chunker, Gemini embedder, language detection
	`vector-store`	Qdrant client wrapper for semantic code search
	`github`	GitHub REST API client (repos, issues, PRs)
	`shared`	Shared TypeScript types across the frontend

Backend layering

The API follows a strict **routes** → **services** → **repositories** → **models** separation. Route handlers contain no business logic. Services include `BrainService`, `IngestionService`, `WorkspaceManager`, `PatchExecutor`, `GitService`, and `GitHubPRService`.

3. The Multi-Agent Pipeline

The heart of Forge is a LangGraph `StateGraph` with conditional feedback loops. A single typed object, `SentinelState`, flows through every node; agents communicate **only** through structured state — never free-form text.

```
START
  → repo_analyzer        understand structure & tech stack
  → issue_analyzer       classify the selected issue
  → retrieve_context     semantic search over Qdrant (top-5 chunks)
  → load_full_files      read complete files from workspace
  → planner              tasks, affected files, dependencies
  → developer            generate unified-diff patches
  → apply_patches  ──┐
  │     patch failed & retries left → developer
  → validator      ──┐
  │     invalid & retries left → developer
  → test_agent           LLM-simulated test verification
  → reviewer       ──┐
  │     rejected & retries left → developer
  → pr_generator         PR title + body draft
END
```

The six core agents

Agent	Role	Structured Output
Issue Analyzer	Classify type, severity, impact, confidence	`IssueAnalysis`
Planner	Identify affected files, build task DAG & strategy	`Plan` (tasks, affected_files, dependencies)
Developer	Generate code changes as unified diffs (never commits)	`list[CodeChange]`
QA / Test	Reason about tests & coverage (LLM-simulated for now)	`list[TestResult]`
Reviewer	Find bugs, security risks, suggest improvements	`Review` (approved, comments, security)
PR Agent	Generate PR title, body, release notes	`PullRequestDraft`

Supporting nodes: `Repo Analyzer`, `Retrieve Context`, `Load Full Files`, and the `Validator` (a lightweight pre-review check).

4. The Auto-Repair Loop

Forge's standout capability is **self-correction**. Three conditional edges route work back to the Developer agent when something fails, bounded by `max_iterations` (default 3):

- **Patch failure** — if a unified diff does not apply cleanly, the full file content plus the failed patch are fed back for regeneration.
- **Validation failure** — the Validator's issues and suggestions are returned to the Developer.
- **Review rejection** — the Reviewer's comments, security concerns, and suggestions trigger a retry.

This feedback is packaged into a `RepairContext` object carrying exactly the information the LLM needs to fix its specific mistakes. Failures are never silently swallowed — they surface in the agent-run status.

5. Implementation Status

The roadmap defines an 11-phase critical path from "PR draft generator" to "autonomous GitHub engineer." **All seven critical-path phases are complete.**

Phase	Deliverable	Status
1	Surface real diffs in UI (`DiffViewer`)	✅ **Done**
2	Repository Workspace (`WorkspaceManager` — clone via PAT)	✅ **Done**
3	Full File Retrieval (`FileLoader`)	✅ **Done**
4	Patch Executor (apply & validate unified diffs)	✅ **Done**
5	Git Operations (`GitService` — branch, commit, push)	✅ **Done**
6	GitHub PR Creation (`GitHubPRService`)	✅ **Done**
7	Auto-Repair Loop (conditional edges)	✅ **Done**
8–9	Real validation & test execution	◷ **Deferred (needs Docker)**
10	Docker sandbox (CPU/mem/network limits)	◷ **Deferred**
11	Observability (OpenTelemetry, token metrics)	◷ **Deferred**

Foundational capabilities (all complete)

Repository ingestion, repository analysis, issue retrieval & selection, context retrieval, implementation planning, patch generation, code review, and PR draft generation are all operational end-to-end.

6. Frontend & Real-Time Experience

The Next.js 15 dashboard is designed to feel like "GitHub × Linear × Cursor × Datadog" — clean, fast, technical, and real-time. Pages include the Dashboard, Repositories (list & detail with file tree + language breakdown), the Agent Center, a live Agent Run Detail view, and Settings.

Recent work on the current branch (`feature/live-agent-outputs`) makes the autonomy **legible**: instead of staring at a spinner while a 30–90s run executes, the user watches each agent's output stream in as a distinct animated card — issue-analysis metadata, planner tasks, syntax-highlighted code diffs, review findings, and the PR draft preview. This builds on the prior "real-time frontend synchronization" work that tracks the live pipeline stage.

New components in flight

- `issue-analysis-card.tsx` — severity, confidence, impact metadata
- `output-card.tsx` — generic animated container for each agent's result
- `pr-draft-card.tsx` — PR title/body preview
- `diff-viewer.tsx` — unified-diff renderer with copy-to-clipboard

7. Key Architectural Decisions

GitHub authentication via Personal Access Token

Write operations use the user's PAT (scope: `repo`), collected at repository connection and stored **encrypted** on the `Repository` model (`github_pat` column, migration 003). It is never logged and never returned to the frontend. GitHub App / OAuth flows are intentionally avoided.

Per-run ephemeral workspaces

Each run clones the target repo once to `/tmp/forge-workspaces/{agent_run_id}/`. Agents read complete files from disk (not the GitHub API) for editing. Cleanup always runs on completion; workspaces are never reused.

Safety constraints

No destructive GitHub actions — no force pushes, no branch deletion. The PAT never enters streamed or persisted graph state. The current MVP targets small static HTML/CSS repositories, with Docker-sandboxed execution deferred until the core pipeline is proven.

8. Data & State Model

Every workflow state is a Pydantic model — raw dictionaries are never passed between nodes. PostgreSQL holds repositories, issues, repository files, ingestion runs, and agent runs (three Alembic migrations). Qdrant holds per-repository embeddings (Gemini `text-embedding-004`) for semantic retrieval. The central `SentinelState` is organized by pipeline phase: Understand → Decide → Retrieve → Plan → Execute → Validate → Ship.

9. What's Next

- **Docker sandbox (Phase 10)** — isolate all build/test/lint execution with CPU, memory, timeout, and network limits before handling untrusted repositories.
- **Real validation & test execution (Phases 8–9)** — replace LLM-simulated checks with `py_compile`, `tsc`, `pytest`, `npm test` inside the sandbox.
- **Observability (Phase 11)** — OpenTelemetry traces, token-usage tracking, run success/failure metrics.
- **Scale beyond static HTML** — expand from the 2–3 file MVP profile to full application codebases.